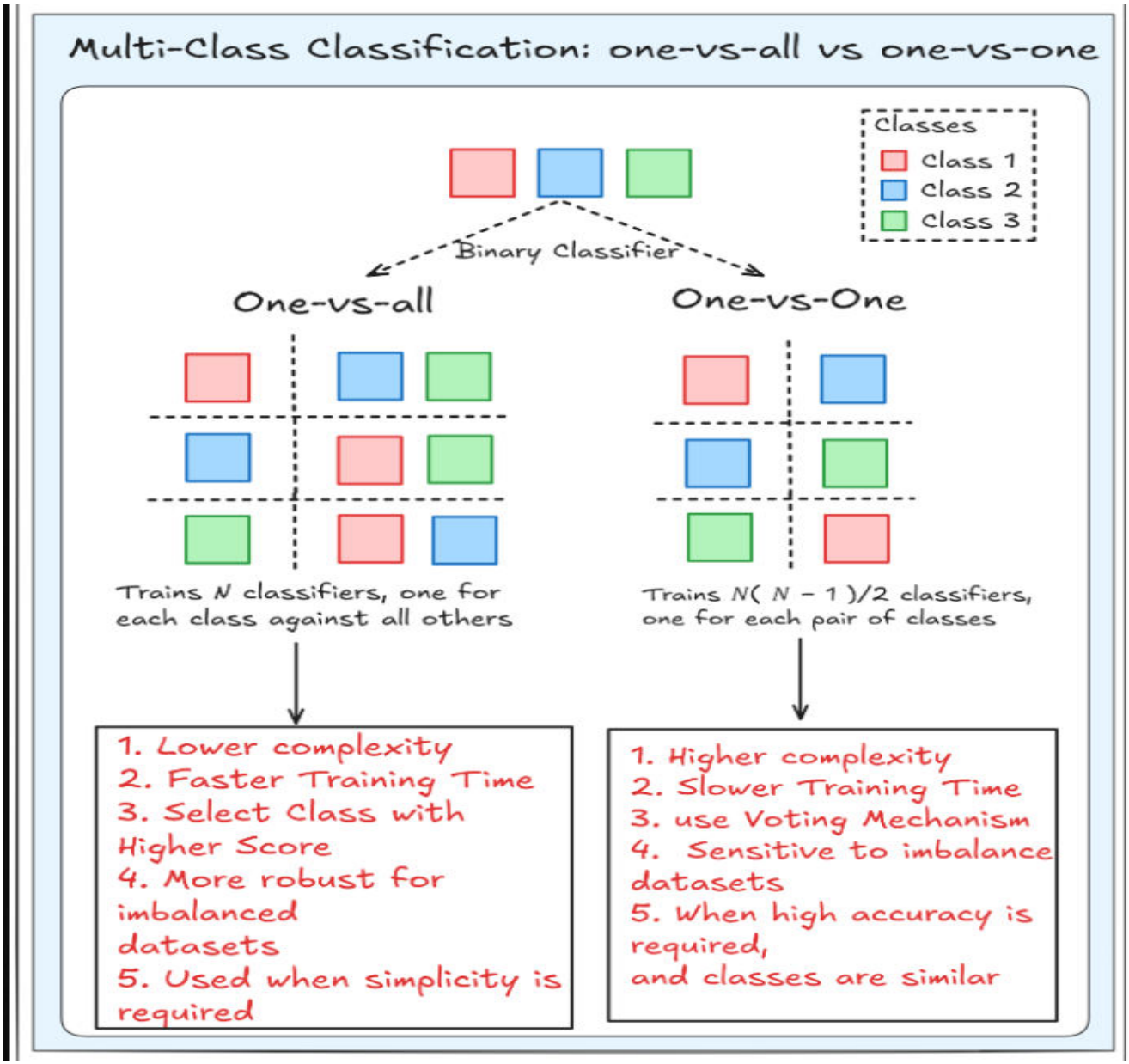
Important Elements of Machine Learning

Multiclass Classification - One-vs-All (OvA)

What is One-vs-All (OvA)?

One-vs-All (also called One-vs-Rest, OvR) is a strategy used for multiclass classification where a classifier is trained for each class against all other classes. This means:

- For **N** classes, **N** binary classifiers are trained.
- Each classifier predicts whether a data point belongs to a specific class or not.
- During inference, the classifier with the highest confidence score determines the final class.



How One-vs-All Works?

1. **Step 1: Convert the Multiclass Problem into Multiple Binary Problems**
   - For **each** class CiC_iCi, assign it as **positive (1)** and all other classes as **negative (0)**.
   - Train a binary classifier on this dataset.

2. **Step 2: Train N Binary Classifiers**
   - If there are **4 classes (A, B, C, D)**, train 4 classifiers:
     - Classifier 1: A vs (B, C, D)
     - Classifier 2: B vs (A, C, D)
     - Classifier 3: C vs (A, B, D)
     - Classifier 4: D vs (A, B, C)

3. **Step 3: Make Predictions**
   - For a new input, each classifier predicts a probability score.
   - The class with the highest probability score is chosen as the final prediction.

Advantages of One-vs-All

✔ Simple and easy to implement.
✔ Works well with any binary classifier.
✔ Efficient for problems with a moderate number of classes.

Disadvantages of One-vs-All

✖ Performance may degrade for a large number of classes.
✖ Can be affected by class imbalance.
✖ Requires multiple classifiers, increasing training time.

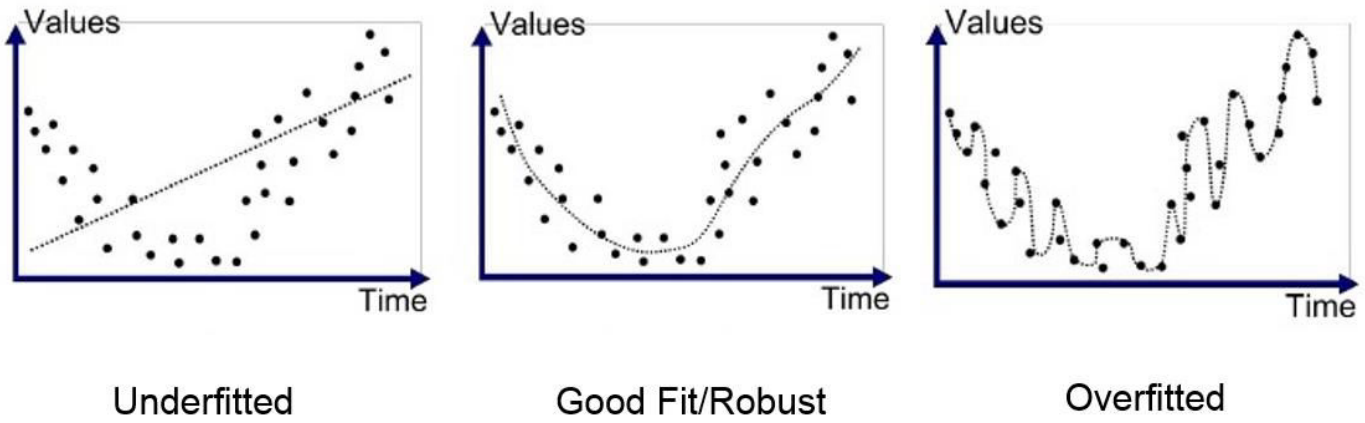
Use Cases

✅ Handwritten digit recognition (0-9).
✅ Sentiment analysis with multiple categories (positive, negative, neutral).
✅ Medical diagnosis (classifying diseases into multiple types).

Overfitting and Underfitting in Machine Learning

What is Overfitting?

Overfitting occurs when a machine learning model learns the **training data too well**, capturing **noise and unnecessary details** rather than general patterns. This results in high accuracy on training data but poor performance on new, unseen data.



Causes of Overfitting

- **Too complex model** (e.g., deep decision trees, high-degree polynomial regression)
- **Insufficient training data**
- **Too many features (high dimensionality)**
- **Lack of regularization**

Symptoms of Overfitting

✔ High accuracy on training data but low accuracy on test/validation data.
✔ Large gap between training and test performance.

How to Prevent Overfitting?

✅**More training data** – Helps generalization.
✅**Feature selection** – Remove irrelevant features.
✅**Regularization** (L1/L2) – Penalizes model complexity.
✅**Cross-validation** – Ensures better model evaluation.
✅**Dropout (for neural networks)** – Reduces reliance on specific neurons.
✅**Early stopping** – Stops training when validation loss increases.

What is Underfitting?

Underfitting occurs when a model is **too simple** to capture the underlying structure of data, leading to poor performance on both training and test data.

Causes of Underfitting

- **Model too simple** (e.g., linear regression for a non-linear problem)
- **Insufficient training time (in neural networks)**
- **Too few features or wrong feature selection**

Symptoms of Underfitting

✔ Low accuracy on both training and test data.
✔ Model fails to capture patterns in data.

How to Fix Underfitting?

✅**Use a more complex model** (e.g., from linear regression to polynomial regression).
✅**Increase training time** (for neural networks).
✅**Feature engineering** – Add more relevant features.
✅**Reduce regularization strength** – Allows model to learn more.

Overfitting vs. Underfitting Summary

Factor	Overfitting	Underfitting
Complexity	Too complex	Too simple
Performance on Training Data	High	Low
Performance on Test Data	Low	Low
Cause	Capturing noise	Not capturing patterns
Solution	Regularization, pruning, more data	More features, complex model, longer training

Error Measures in Machine Learning

Error measures are used to evaluate the performance of machine learning models. They help determine how well a model is making predictions. The type of error measure used depends on whether the task is **regression** or **classification**.

1. Error Measures for Regression

Regression problems deal with **continuous values**, so error measures focus on how far predictions are from actual values.

1.1 Mean Absolute Error (MAE)

Formula:

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Interpretation:

- Measures the average absolute difference between actual and predicted values.
- Less sensitive to outliers compared to MSE.

✔ Pros: Easy to interpret.
✖ Cons: Doesn't penalize large errors more than small ones.

1.2 Mean Squared Error (MSE)

Formula:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Interpretation:

- Penalizes larger errors more due to squaring.
- Used in optimization (e.g., gradient descent).

✔ Pros: Good for optimization.
✖ Cons: Sensitive to outliers.

1.3 Root Mean Squared Error (RMSE)

Formula:

$$RMSE = \sqrt{MSE}$$

Interpretation:

- Square root of MSE to bring error back to original units.
- More interpretable than MSE.

✔ Pros: Same unit as target variable.
✖ Cons: Sensitive to large errors.

1.4 R^2 Score (Coefficient of Determination)

Formula:

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

Interpretation:

- Measures how well the model explains variance in data.

- **Value close to 1**→ Good fit, **close to 0**→ Poor fit.

✔ Pros: Explains model's goodness-of-fit.
✖ Cons: Can be misleading if used alone.

2. Error Measures for Classification

Classification problems deal with **categorical labels**, so different metrics are used.

2.1 Accuracy

Formula:

$$Accuracy = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

Interpretation:

- Measures overall correctness.
- Can be misleading in **imbalanced datasets**.

✔ Pros: Simple to understand.
✖ Cons: Not useful when classes are imbalanced.

2.2 Precision

Formula:

$$Precision = \frac{TP}{TP + FP}$$

Interpretation:

- Measures how many predicted **positives** are actually **positive**.
- High precision = Fewer false positives.

✔ Pros: Useful when false positives are costly (e.g., spam detection).
✖ Cons: Does not consider false negatives.

2.3 Recall (Sensitivity)

Formula:

$$Recall = \frac{TP}{TP + FN}$$

Interpretation:

- Measures how many actual **positives** were correctly predicted.
- High recall = Fewer false negatives.

✔ Pros: Important in medical diagnosis.
✖ Cons: May increase false positives.

2.4 F1-Score

Formula:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Interpretation:

- **Harmonic mean** of precision and recall.
- Used when **false positives and false negatives are equally important**.

✔ Pros: Balances precision and recall.
✖ Cons: Doesn't consider true negatives.

2.5 ROC-AUC (Receiver Operating Characteristic - Area Under Curve)

- **ROC Curve**: Plots **True Positive Rate (TPR) vs. False Positive Rate (FPR)**.
- **AUC (Area Under Curve)**: Measures how well a model separates classes.

✔ Pros: Works well with imbalanced data.
✖ Cons: More complex to interpret.

Dimensionality Reduction Techniques – PCA

Dimensionality reduction is the process of reducing the number of features (variables) in a dataset while retaining as much meaningful information as possible. It is crucial for improving model performance, reducing computation time, and removing noise from data.

1. Principal Component Analysis (PCA)

PCA is a statistical technique used to transform high-dimensional data into a lower-dimensional space while preserving the variance of the original dataset. It finds a new set of axes (principal components) that maximize variance and project the data onto these axes.

📌 **Steps in PCA**

Step 1: Standardize the Data

Since PCA is affected by different scales of features, the data should be standardized to have zero mean and unit variance. This ensures that all features contribute equally.

Formula for standardization:

$$X' = \frac{X - \mu}{\sigma}$$

where X is the original feature, μ is the mean, and σ is the standard deviation.

◆ **Python Code Example**

```python
from sklearn.preprocessing import StandardScaler
import numpy as np

X = np.array([[2.5, 3.1], [1.2, 0.8], [4.5, 5.2], [3.2, 3.8]])
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Step 2: Compute the Covariance Matrix

The covariance matrix represents how features vary together. It is calculated as:

$$C = \frac{1}{n-1} X^T X$$

where X is the standardized data matrix.

◆ **Python Code Example**

```python
cov_matrix = np.cov(X_scaled.T)
print(cov_matrix)
```

Step 3: Compute Eigenvalues and Eigenvectors

Eigenvalues measure the variance explained by each principal component, while eigenvectors define the direction of these components.

◆ **Python Code Example**

```python
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)
print("Eigenvalues:", eigenvalues)
print("Eigenvectors:\n", eigenvectors)
```

Step 4: Select Top k Principal Components

Sort the eigenvalues in descending order and select the top k components that explain the most variance. The explained variance ratio is given by:

$$\text{Explained Variance} = \frac{\lambda_i}{\sum \lambda}$$

where λi are the eigenvalues.

◆ **Python Code Example**

```python
explained_variance_ratio = eigenvalues / np.sum(eigenvalues)
print("Explained Variance Ratio:", explained_variance_ratio)
```

Step 5: Project Data onto the New Feature Space

The original data is transformed using the selected eigenvectors to obtain reduced dimensions.

🔷 Python Code Example

```
# Project onto first two principal components
X_pca = np.dot(X_scaled, eigenvectors[:, :2])
print(X_pca)
```

📌 Variants of PCA

Sparse PCA

- Introduces sparsity constraints on principal components to select fewer influential features.
- Useful when dealing with highly correlated features.
  🔷 **Use Case**: Gene expression data, where only a few genes contribute to a condition.

```python
from sklearn.decomposition import SparsePCA
sparse_pca = SparsePCA(n_components=2)
X_sparse_pca = sparse_pca.fit_transform(X_scaled)
```

Kernel PCA

- Extends PCA to non-linear data using kernel functions (e.g., polynomial, RBF, sigmoid).
  🔷 **Use Case**: Facial recognition, where data is non-linearly distributed.

```python
from sklearn.decomposition import KernelPCA
kpca = KernelPCA(n_components=2, kernel='rbf')
X_kpca = kpca.fit_transform(X_scaled)
```

Statistical learning approaches

Statistical learning approaches are a set of methods and techniques used to understand and model data by applying statistical principles. These approaches are widely used in machine learning, data analysis, and predictive modeling. They provide a theoretical foundation for understanding how models learn from data and make predictions. Below is an overview of key statistical learning approaches, their types, and their applications.

1. Types of Statistical Learning Approaches

Statistical learning approaches can be broadly categorized into two types:

A. Supervised Learning

- **Definition:** The model learns from labeled data, where the input features (X) and the corresponding output labels (Y) are provided.
- **Goal:** To learn a mapping from inputs to outputs that can generalize to unseen data.
- **Examples:**
  - Regression: Predicting continuous values (e.g., house prices).
  - Classification: Predicting categorical labels (e.g., spam or not spam).

B. Unsupervised Learning

- **Definition:** The model learns from unlabeled data, where only the input features (X) are provided.
- **Goal:** To discover hidden patterns, structures, or relationships in the data.
- **Examples:**
  - Clustering: Grouping similar data points (e.g., customer segmentation).
  - Dimensionality Reduction: Reducing the number of features while preserving important information (e.g., PCA, t-SNE).

2. Key Concepts in Statistical Learning

A. Model Representation

- A model is a mathematical representation of the relationship between input features (X) and output labels (Y).

Example: In linear regression, the model is represented as

$$Y = \beta_0 + \beta_1 X + \epsilon,$$

where β0 and β1 are parameters, and ϵ is the error term.

B. Loss Function

- A loss function measures how well the model's predictions match the true labels.
- Example: Mean Squared Error (MSE) for regression, Cross-Entropy Loss for classification.

C. Optimization

- The process of finding the model parameters that minimize the loss function.
- Example: Gradient Descent, Maximum Likelihood Estimation (MLE).

D. Regularization

- Techniques used to prevent overfitting by adding a penalty term to the loss function.
- Example: L1 regularization (Lasso), L2 regularization (Ridge).

E. Bias-Variance Tradeoff

- Balancing the model's complexity to ensure it generalizes well to unseen data.

3. Common Statistical Learning Methods

A. Linear Models

- **Linear Regression:**
  - Models the relationship between input features and a continuous output.
  - Example: Predicting house prices based on features like size and location.
- **Logistic Regression:**
  - Models the probability of a binary outcome.
  - Example: Predicting whether an email is spam or not.

B. Tree-Based Methods

- **Decision Trees:**
  - Splits the data into branches based on feature values to make predictions.
  - Example: Classifying whether a customer will buy a product.
- **Random Forests:**

- An ensemble of decision trees to improve accuracy and reduce overfitting.

- **Gradient Boosting Machines (GBM):**
  - Sequentially builds trees to correct errors from previous trees.

C. Support Vector Machines (SVM)

- Finds the optimal hyperplane that separates data points of different classes.
- Can be used for both classification and regression.

D. Clustering

- **K-Means:**
  - Groups data into k clusters based on feature similarity.
- **Hierarchical Clustering:**
  - Builds a tree-like structure of clusters.

E. Dimensionality Reduction

- **Principal Component Analysis (PCA):**
  - Reduces the number of features while preserving variance.
- **t-SNE:**
  - Visualizes high-dimensional data in 2D or 3D.

F. Neural Networks

- Models complex relationships using layers of interconnected nodes (neurons).
- Example: Deep learning for image recognition.

4. Applications of Statistical Learning

A. Predictive Modeling

- Forecasting future outcomes based on historical data.
- Example: Predicting stock prices, customer churn, or disease risk.

B. Pattern Recognition

- Identifying patterns in data.
- Example: Facial recognition, speech recognition.

C. Natural Language Processing (NLP)

- Analyzing and generating human language.
- Example: Sentiment analysis, machine translation.

D. Recommendation Systems

- Suggesting products or content based on user behavior.

- Example: Netflix recommendations, Amazon product suggestions.

E. Anomaly Detection

- Identifying unusual patterns or outliers.
- Example: Fraud detection, network intrusion detection.

5. Advantages of Statistical Learning Approaches

- **Theoretical Foundation:** Provides a rigorous framework for understanding data and models.
- **Interpretability:** Many statistical models (e.g., linear regression) are easy to interpret.
- **Flexibility:** Can be applied to a wide range of problems and data types.
- **Robustness:** Techniques like regularization and cross-validation improve model performance.

6. Challenges in Statistical Learning

- **Overfitting:** Models may perform well on training data but poorly on unseen data.
- **Curse of Dimensionality:** Performance degrades as the number of features increases.
- **Data Quality:** Noisy or incomplete data can lead to poor model performance.
- **Computational Complexity:** Some methods (e.g., deep learning) require significant computational resources.

7. Tools and Libraries for Statistical Learning

- **Python:**
  - Scikit-learn: For traditional machine learning algorithms.
  - Statsmodels: For statistical modeling and hypothesis testing.
  - TensorFlow/PyTorch: For deep learning.

Introduction to Scikit-Learn (sklearn)

Scikit-Learn (**sklearn**) is a widely used Python library for machine learning. It provides a comprehensive set of tools for data preprocessing, model training, evaluation, and selection. The framework is built on top of NumPy, SciPy, and Matplotlib, making it efficient and easy to integrate into data science projects.

Why Use Scikit-Learn?

Scikit-learn is preferred because:
✔ It supports a wide range of **machine learning algorithms** (classification, regression, clustering, dimensionality reduction).
✔ It provides **preprocessing tools** for handling missing data, feature scaling, and encoding categorical variables.

✔ It includes utilities for **model selection**, such as cross-validation and hyperparameter tuning.
✔ It offers **evaluation metrics** to assess model performance.

Installation

```bash
pip install scikit-learn
```

To check if it's installed, run:

```python
import sklearn
print(sklearn.__version__)
```

Key Features and Modules in Scikit-Learn

1. Data Preprocessing

Before training a machine learning model, we need to preprocess the data to ensure it's clean and structured properly.

Feature Scaling (Standardization & Normalization)

Most machine learning models perform better when features have similar scales.

- **Standardization:** Transforms data to have mean = 0 and standard deviation = 1.
- **Normalization:** Scales values between 0 and 1.

Example:

```python
from sklearn.preprocessing import StandardScaler, MinMaxScaler

scaler = StandardScaler()  # Standardization
X_scaled = scaler.fit_transform(X)

minmax_scaler = MinMaxScaler()  # Normalization
X_normalized = minmax_scaler.fit_transform(X)
```

Encoding Categorical Data

Machine learning models work with numerical data, so categorical variables must be converted.

Example using **One-Hot Encoding**:

```python
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(sparse_output=False)  # Convert categorical to numerical
X_encoded = encoder.fit_transform(X_categorical)
```

Handling Missing Data

Missing values can be replaced using mean, median, or mode.

```python
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="mean")  # Replace missing values with the mean
X_imputed = imputer.fit_transform(X)
```

Splitting Data for Training and Testing

Dividing data into training and testing sets ensures that we evaluate models on unseen data.

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

2. Machine Learning Models in Scikit-Learn

Scikit-learn provides a variety of algorithms for both supervised and unsupervised learning.

Supervised Learning Algorithms

These algorithms learn from labeled data.

Linear Regression (For Continuous Predictions)

Used for predicting numeric values

```python
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Logistic Regression (For Classification Problems)

Used when predicting categorical values (e.g., spam or not spam).

```python
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Decision Tree Classifier

Used for both classification and regression.

```python
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Random Forest Classifier

A collection of decision trees used for better accuracy.

```python
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Unsupervised Learning Algorithms

These algorithms find patterns in **unlabeled** data.

K-Means Clustering

Groups data into clusters.

```python
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=3)
kmeans.fit(X)
clusters = kmeans.predict(X)
```

Principal Component Analysis (PCA) - Dimensionality Reduction

Reduces the number of features while keeping important information.

```python
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
```

3. Model Evaluation and Metrics

After training a model, we need to evaluate how well it performs.

Classification Metrics

Used for evaluating classification models.

Confusion Matrix:

```python
from sklearn.metrics import confusion_matrix

print(confusion_matrix(y_test, y_pred))
```

Precision, Recall, and F1-score:

```python
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

Regression Metrics

Used for evaluating regression models.

Mean Absolute Error (MAE), Mean Squared Error (MSE), R^2 Score:

```python
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

print("MAE:", mean_absolute_error(y_test, y_pred))
print("MSE:", mean_squared_error(y_test, y_pred))
print("R² Score:", r2_score(y_test, y_pred))
```

4. Model Selection and Hyperparameter Tuning

Finding the best model requires tuning hyperparameters.

Cross-validation:

```python
from sklearn.model_selection import cross_val_score

scores = cross_val_score(model, X, y, cv=5)
print("Cross-validation scores:", scores)
```

Grid Search for Hyperparameter Tuning:

```
from sklearn.model_selection import GridSearchCV

param_grid = {'n_estimators': [10, 50, 100]}
grid_search = GridSearchCV(RandomForestClassifier(), param_grid, cv=5)
grid_search.fit(X_train, y_train)

print("Best Parameters:", grid_search.best_params_)
```

Scikit-learn simplifies the entire machine learning pipeline, from **data preprocessing** to **model training and evaluation**. It is widely used because of its efficiency and ease of implementation.

Interview questions

Multiclass Classification

Q1. What is multiclass classification, and how does it differ from binary classification?
A1. Multiclass classification is a type of classification where there are more than two possible classes for a given input. Unlike binary classification, where the model predicts between two labels (e.g., 0 or 1), multiclass classification assigns an input to one of three or more categories (e.g., class A, B, or C).

Q2. What are common algorithms used for multiclass classification?
A2.

- Logistic Regression (One-vs-All)
- Decision Trees
- Random Forest
- Support Vector Machines (One-vs-One or One-vs-All)
- Neural Networks
- Naïve Bayes

Q3. How do you handle imbalanced data in a multiclass classification problem?
A3.

- Use oversampling (SMOTE) or undersampling techniques.
- Assign class weights in algorithms (e.g., class_weight='balanced' in scikit-learn).
- Try different evaluation metrics (F1-score, ROC-AUC).
- Use ensemble methods like boosting.

One-vs-All (OvA) Strategy

Q4. What is the One-vs-All (OvA) approach in multiclass classification?
A4. The One-vs-All (OvA), or One-vs-Rest (OvR), strategy involves training multiple binary classifiers, each distinguishing one class from all the others. The final prediction is based on the classifier with the highest probability score.

Q5. What are the advantages and disadvantages of using One-vs-All?
A5.
✅ Advantages:

- Simple to implement.
- Works well when one class is dominant.
- Scales well with many classes.

❌ Disadvantages:

- Requires training multiple models (one for each class).
- Can lead to misclassification if classifiers overlap.

Overfitting and Underfitting

Q6. Define overfitting and underfitting. How can they be prevented?
A6.

- **Overfitting**: The model learns noise from training data and performs poorly on unseen data.
  Prevention: Regularization (L1/L2), pruning (for trees), dropout (for neural networks), reducing complexity.
- **Underfitting**: The model is too simple and fails to capture patterns.
  Prevention: Adding more features, increasing model complexity, reducing regularization.

Q7. How can cross-validation help in preventing overfitting?
A7. Cross-validation, such as k-fold, ensures that the model is trained and tested on different subsets of data, reducing the risk of overfitting to a specific dataset.

Error Measures for Multiclass Classification

Q8. What are the common error measures for evaluating a multiclass classification model?
A8.

- **Accuracy**: Correct PredictionsTotal Predictions\frac{\text{Correct Predictions}}{\text{Total Predictions}}Total PredictionsCorrect Predictions
- **Precision, Recall, F1-score** (per class and weighted average)
- **Confusion Matrix**: Shows true positives, false positives, false negatives, and true negatives for all classes.
- **Logarithmic Loss (Log Loss)**: Penalizes incorrect confident predictions.
- **ROC-AUC for Multiclass** (One-vs-All approach).

Q9. When should you prefer F1-score over accuracy?
A9. When dealing with imbalanced datasets, F1-score provides a better balance between precision and recall.

PCA (Principal Component Analysis) Learning

Q10. What is PCA, and how does it help in machine learning?
A10. PCA is a dimensionality reduction technique that transforms high-dimensional data into a lower-dimensional space while preserving maximum variance. It helps reduce computational complexity and removes noise.

Q11. How do you decide the number of principal components to retain in PCA?
A11. Use the explained variance ratio. Typically, we select components that preserve at least **95% of the variance**.

Q12. Can PCA be used for classification tasks?
A12. PCA itself is not a classifier, but it helps by reducing feature dimensions before applying classification models.

Statistical Learning Approaches

Q13. What is the difference between statistical learning and machine learning?
A13.

- **Statistical learning** focuses on inference, probability distributions, and mathematical theory.
- **Machine learning** is more about optimization and prediction using large datasets.

Q14. What are some key statistical learning techniques used in classification?
A14.

- **Bayesian Classifiers** (e.g., Naïve Bayes).
- **Linear Discriminant Analysis (LDA)**.
- **Maximum Likelihood Estimation (MLE)**.

Introduction to Scikit-Learn (sklearn) Framework

Q15. What is Scikit-Learn, and why is it popular?
A15. Scikit-Learn (sklearn) is a Python library for machine learning, offering simple APIs for classification, regression, clustering, and preprocessing. It is popular due to its ease of use and efficiency.

Q16. How do you train a multiclass classifier using Scikit-Learn?
A16. Example using LogisticRegression:

```
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Load dataset
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train classifier
clf = LogisticRegression(multi_class='ovr', solver='liblinear')
clf.fit(X_train, y_train)

# Predict
y_pred = clf.predict(X_test)
```

Q17. What is the difference between One-vs-All and One-vs-One strategies in Scikit-Learn?
A17.

- **One-vs-All (ovr)**: Trains one classifier per class vs. rest.
- **One-vs-One (ovo)**: Trains classifiers for every possible pair of classes.

Q18. How can you evaluate a classification model in Scikit-Learn?
A18. Using classification_report:

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

Softmax Regression, One-vs-All, One-vs-One

https://www.kaggle.com/code/brittasmith/softmax-regression-one-vs-all-one-vs-one

Exercise: Overfitting and Underfitting

https://www.kaggle.com/code/berkayalan/exercise-overfitting-and-underfitting

Machine Learning Model Evaluation and Error Metric

arrow_drop_up2

https://www.kaggle.com/code/viveknimsarkar/machine-learning-model-evaluation-and-error-metric

Principal Component Analysis

https://www.kaggle.com/code/ryanholbrook/principal-component-analysis

https://www.kaggle.com/code/nirajvermafcb/principal-component-analysis-explained

https://www.kaggle.com/code/nirajvermafcb/principal-component-analysis-with-scikit-learn

Statistical Learning Tutorial for Beginners

https://www.kaggle.com/code/kanncaa1/statistical-learning-tutorial-for-beginners

https://www.kaggle.com/code/hilalmleykeyuksel/statistical-learning-tutorial

Tutorial scikit-learn || Zero to Hero

https://www.kaggle.com/code/rachidyz/tutorial-scikit-learn-zero-to-hero-part-i